Práctica RA2 parte 1

Sergio Díaz González y José A. Santos García

21 de octubre 2026

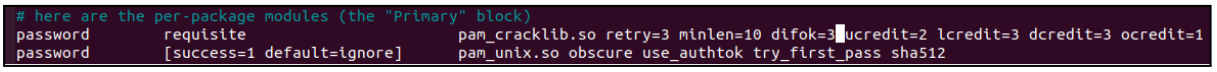


ÍNDICE

Práctica 1: Instalación de librería libpam_cracklib....3
Parte 1: Política de contraseñas....3
Parte 2: Creación de usuario....3
Práctica 2: Proceso de autenticación en 2 pasos....4
Parte 1: Configuración de Google Authenticator en una máquina virtual Linux y configuración de servicio SSH....4
1.3.- Preparación de la protección del servicio SSH....4
1.4.- Habilitar PAM en la configuración SSH....5
Parte 2: Activación del 2FA por parte del usuario con Google Authenticator (máquina virtual) y Microsoft Authenticator (móvil)....6
2.1.- Activación del 2FA....6
2.2.- Generación del código QR en formato PNG....7
Prueba de acceso por SSH a localhost con doble verificación....8
Práctica 3: Configuración de ACLs en una máquina Linux con SETFACL....9
Práctica 4: Configuración de ACLs en un router con Packet Tracer....9
2. Configuración de la red en Packet Tracer....9
3. Configuración básica del router....10
4. y 5. Configuración de la ACL y aplicación a una interfaz....10
6. Prueba de la configuración....11
Práctica 5: Cómo montar un RAID 5 en una máquina virtual usando mdadm desde Ubuntu....12
2. Instalación de mdadm....12
3. Identificar los discos....13
4. Configuración de los discos....14
5. Configuración de mdadm....15
6. Verificación del estado del RAID....15
7. Formateo del dispositivo lógico /dev/md/dm0....15
8. Creación del disco spare....16
9. Fallo de un disco del RAID....17
Práctica 6: Copias de seguridad con duplicity....18
2.1.- Creación de los archivos para realizar el backup....18
4.- Realización de la primera copia de seguridad COMPLETA....18
5.- Creación de copias de seguridad INCREMENTALES....19
6.- Restauración de datos....19
7.- Copias EXCLUYENDO el contenido....20

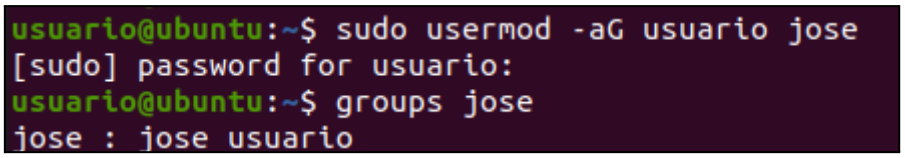
Práctica 1: Instalación de librería libpam_cracklib

Parte 1: Política de contraseñas

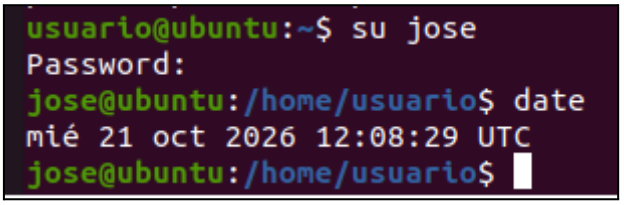


Editamos el archivo */etc/pam.d/common-password* para que pam_cracklib aplique nuestra política de contraseñas. Dejamos la línea con *retry=3*, *minlen=10*, *difok=3*, *ucredit=2*, *lcredit=3*, *dcredit=3* y *ocredit=1*, es decir: mínimo 10 caracteres, al menos 3 diferencias respecto a la contraseña anterior, 2 mayúsculas, 3 minúsculas, 3 dígitos y 1 símbolo. Esta captura muestra el archivo ya modificado.

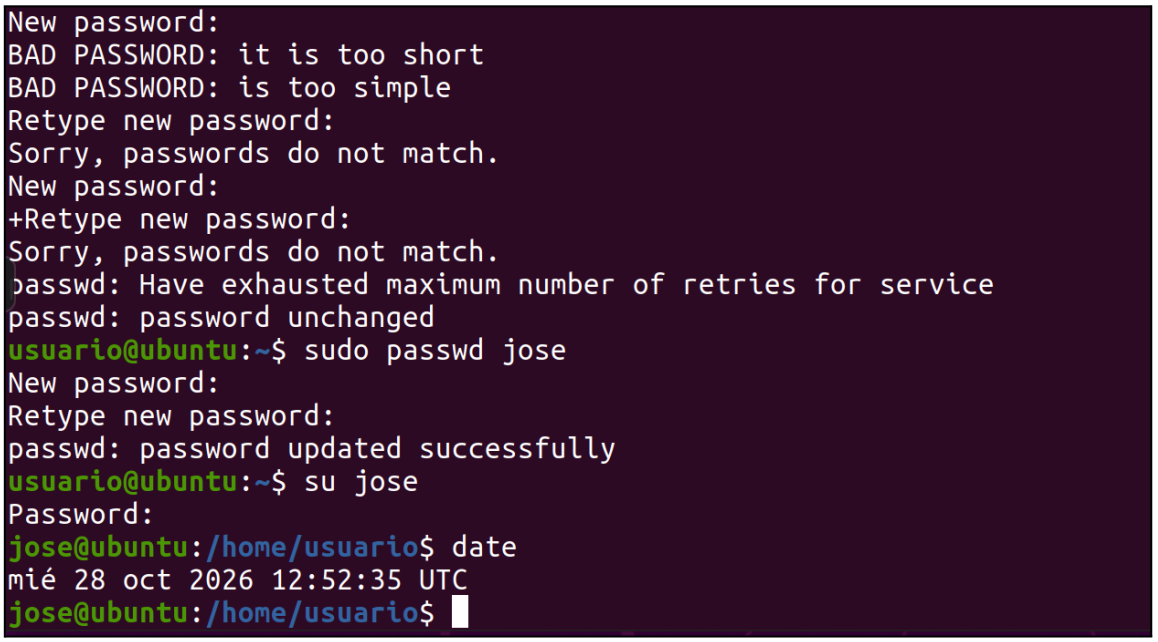
Parte 2: Creación de usuario



Una vez creado el usuario jose, lo añadimos al grupo «usuario» con *sudo usermod -aG usuario jose*. Con *groups jose* comprobamos que ya pertenece a los dos grupos: jose y usuario.



Cambiamos al nuevo usuario con *su jose* y ejecutamos *date* para comprobar que entramos correctamente.



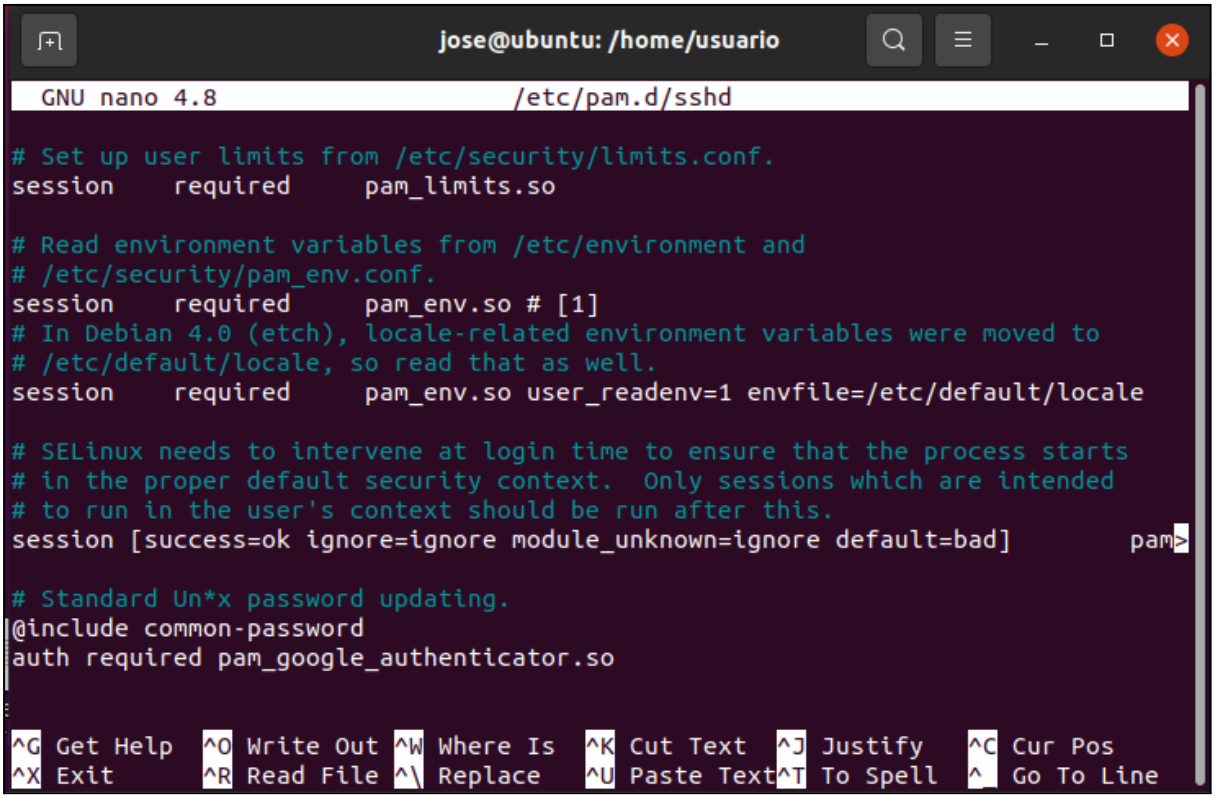
Probamos a ponerle contraseña a jose con *sudo passwd jose*. Al principio el sistema nos rechazó las contraseñas por ser demasiado cortas y simples (it is too short, is too simple), lo que

confirma que la política de pam_cracklib ya se está aplicando; al agotar los intentos la contraseña no cambió. Repetimos el comando con una contraseña que cumple los requisitos y esta vez se actualizó correctamente. Después entramos con *su jose* y ejecutamos *date*.

Práctica 2: Proceso de autenticación en 2 pasos

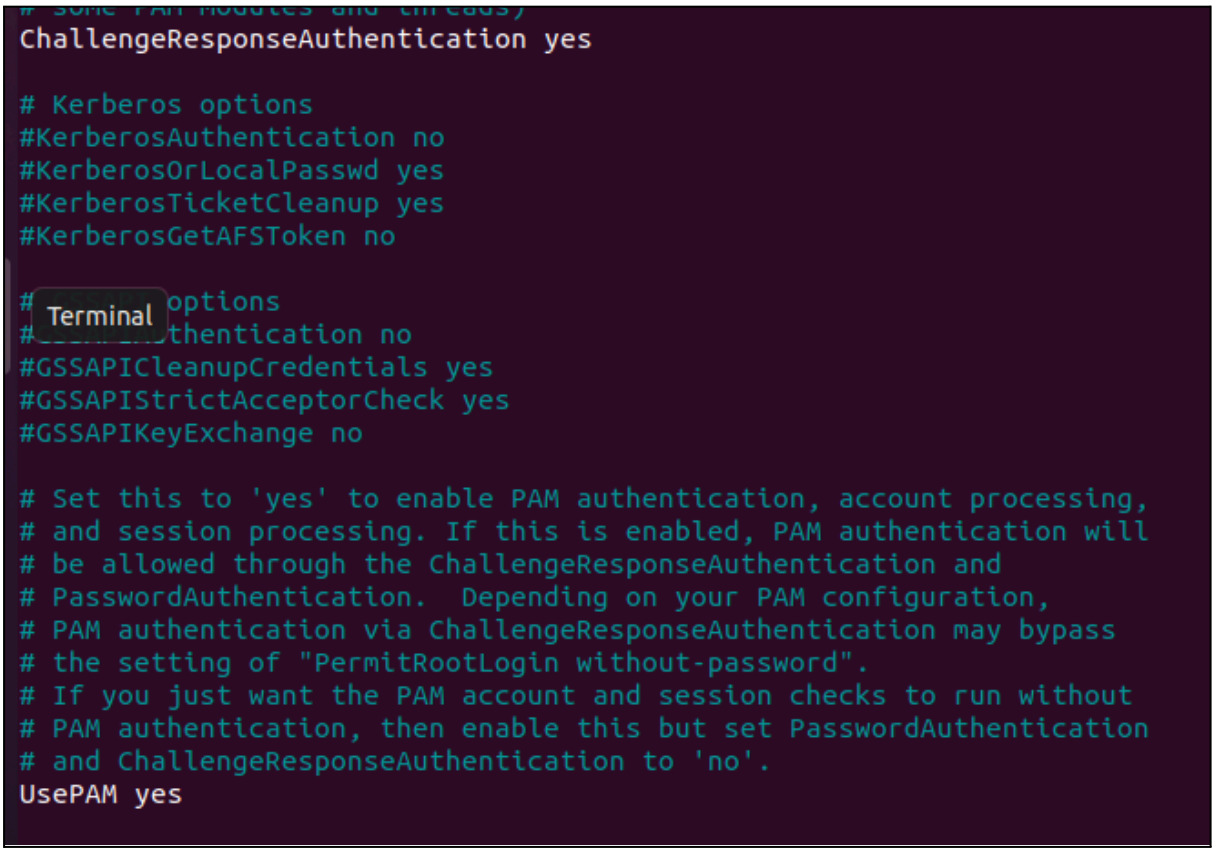
Parte 1: Configuración de Google Authenticator en una máquina virtual Linux y configuración de servicio SSH

1.3.- Preparación de la protección del servicio SSH



Abrimos */etc/pam.d/sshd* con nano y añadimos al final la línea *auth required pam_google_authenticator.so*. Con esto le indicamos a PAM, que es quien gestiona la autenticación en Linux, que use la librería de Google Authenticator cuando alguien se conecte por SSH.

1.4.- Habilitar PAM en la configuración SSH



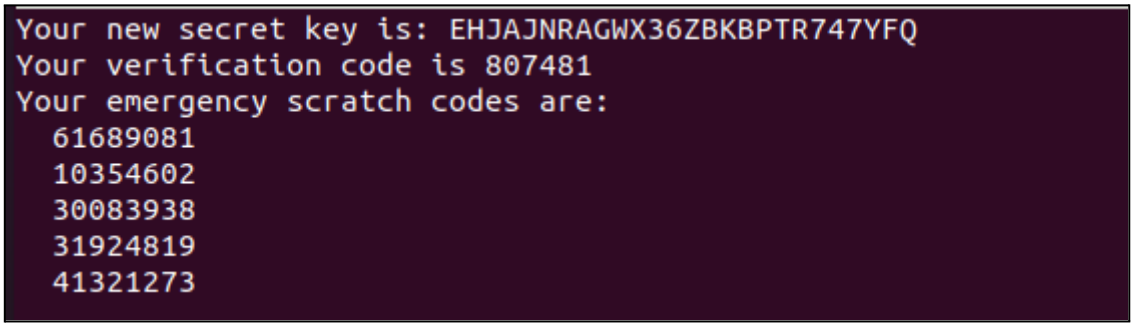
En */etc/ssh/sshd_config* comprobamos que estaban las líneas *ChallengeResponseAuthentication yes* y *UsePAM yes*, para que SSH use PAM en la autenticación.

Parte 2: Activación del 2FA por parte del usuario con Google Authenticator (máquina virtual) y Microsoft Authenticator (móvil)

2.1.- Activación del 2FA

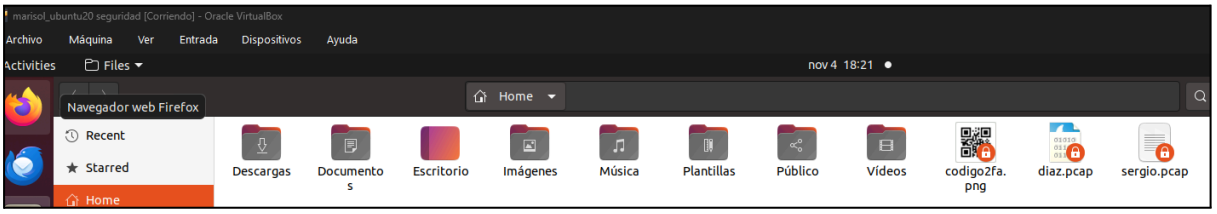


Con el usuario jose ejecutamos *google-authenticator* y contestamos que sí a que los tokens se basen en el tiempo. Nos aparece la URL del código QR (la que empieza por *https://www.google.com*) y el propio QR en la terminal.



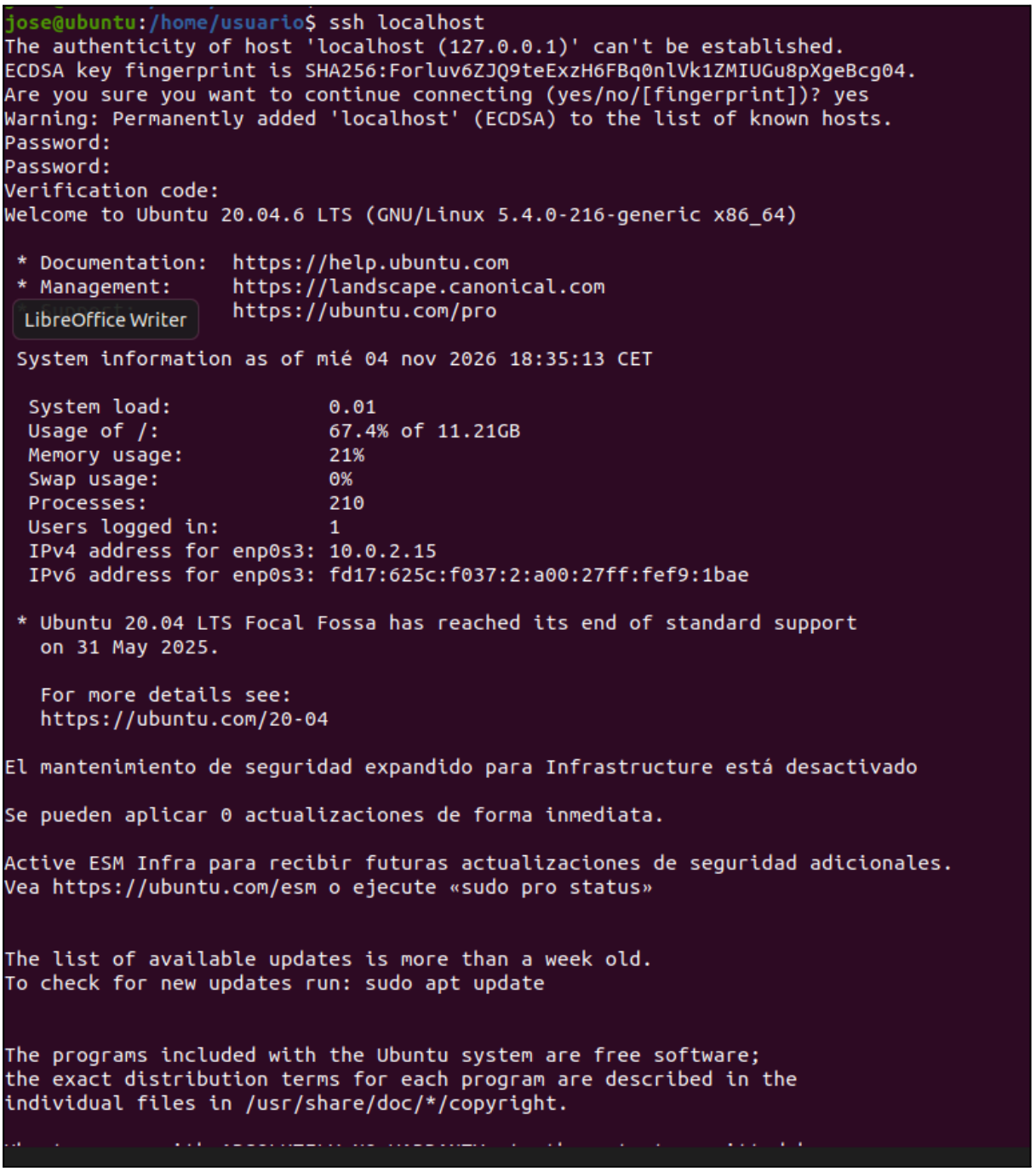
Más abajo del mismo comando salen la llave secreta, el código de verificación y los códigos de emergencia. La llave secreta es la que usamos para añadir la cuenta en la aplicación del móvil.

2.2.- Generación del código QR en formato PNG



Con *qrencode*, que habíamos instalado en el paso 1.2, convertimos la URL del código QR en una imagen PNG. En la carpeta Home aparece el archivo *codigo2fa.png*, que podemos escanear con Microsoft Authenticator desde el móvil.

Prueba de acceso por SSH a localhost con doble verificación



Probamos a conectarnos por SSH a nuestra propia máquina con *ssh localhost*. Primero nos pide la contraseña del usuario (algo que sabemos) y después el código de verificación (Verification code) que genera la aplicación del móvil (algo que tenemos). Al introducir los dos correctamente conseguimos entrar al sistema.

Práctica 3: Configuración de ACLs en una máquina Linux con SETFACL

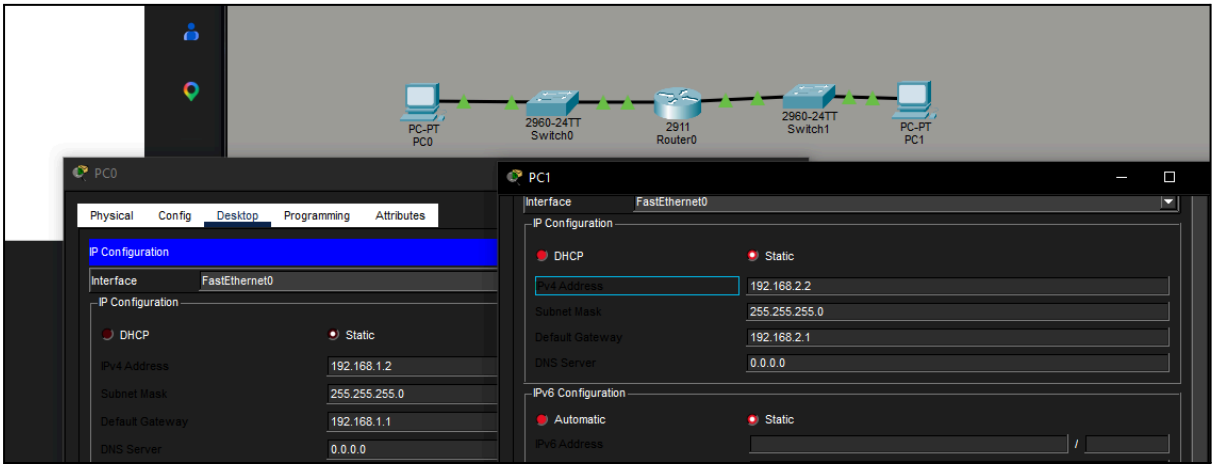
```
jose@ubuntu:/home/usuario$ sudo setfacl -m u::rwx,g:usuario:rwx prueba.txt
jose@ubuntu:/home/usuario$ getfacl prueba.txt
# file: prueba.txt
# owner: jose
# group: jose
user::rwx
group::r--
group:usuario:rwx
mask::rwx
other::r--

jose@ubuntu:/home/usuario$
```

Con *sudo setfacl -m u::rwx,g:usuario:rwx prueba.txt* le dimos al grupo «usuario» permisos de lectura, escritura y ejecución sobre el archivo *prueba.txt*. Con *getfacl prueba.txt* vemos el resultado: el propietario (jose) mantiene rwx y aparece la entrada group:usuario:rwx en la ACL.

Práctica 4: Configuración de ACLs en un router con Packet Tracer

2. Configuración de la red en Packet Tracer



Montamos en Packet Tracer el esquema que piden las instrucciones: un router 2911 en el centro con dos redes conectadas, cada una con su switch 2960 y un PC. La Red 1 es la 192.168.1.0*/24* (PC0 con la IP 192.168.1.2 y puerta de enlace 192.168.1.1) y la Red 2 es la 192.168.2.0*/24* (PC1 con la IP 192.168.2.2 y puerta de enlace 192.168.2.1). Configuramos las IP de los PCs de forma estática, dentro del rango de su red, y como puerta de enlace pusimos la IP de la interfaz del router de cada red.

3. Configuración básica del router

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#inetr
Router(config)#inter
Router(config)#interface gig
Router(config)#interface gigabitEthernet 0/0
Router(config)#interface gigabitEthernet/0
Router(config)#interface gigabitEthernet0/0
Router(config-if)#ip address 192.168.1.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router(config-if)#exit
Router(config)#interf
Router(config)#interface gig0/1
Router(config-if)#ip address 192.168.2.1 255.255.255.0
Router(config-if)#no shutdown
```

Desde la CLI del router configuramos las dos interfaces: gigabitEthernet0*/0* con la IP 192.168.1.1 255.255.255.0 y gig0*/1* con la 192.168.2.1 255.255.255.0, activándolas con *no shutdown*. Se ve que tardamos algunos intentos en escribir bien el nombre de la interfaz, pero al final la gigabitEthernet0*/0* cambia su estado a up.

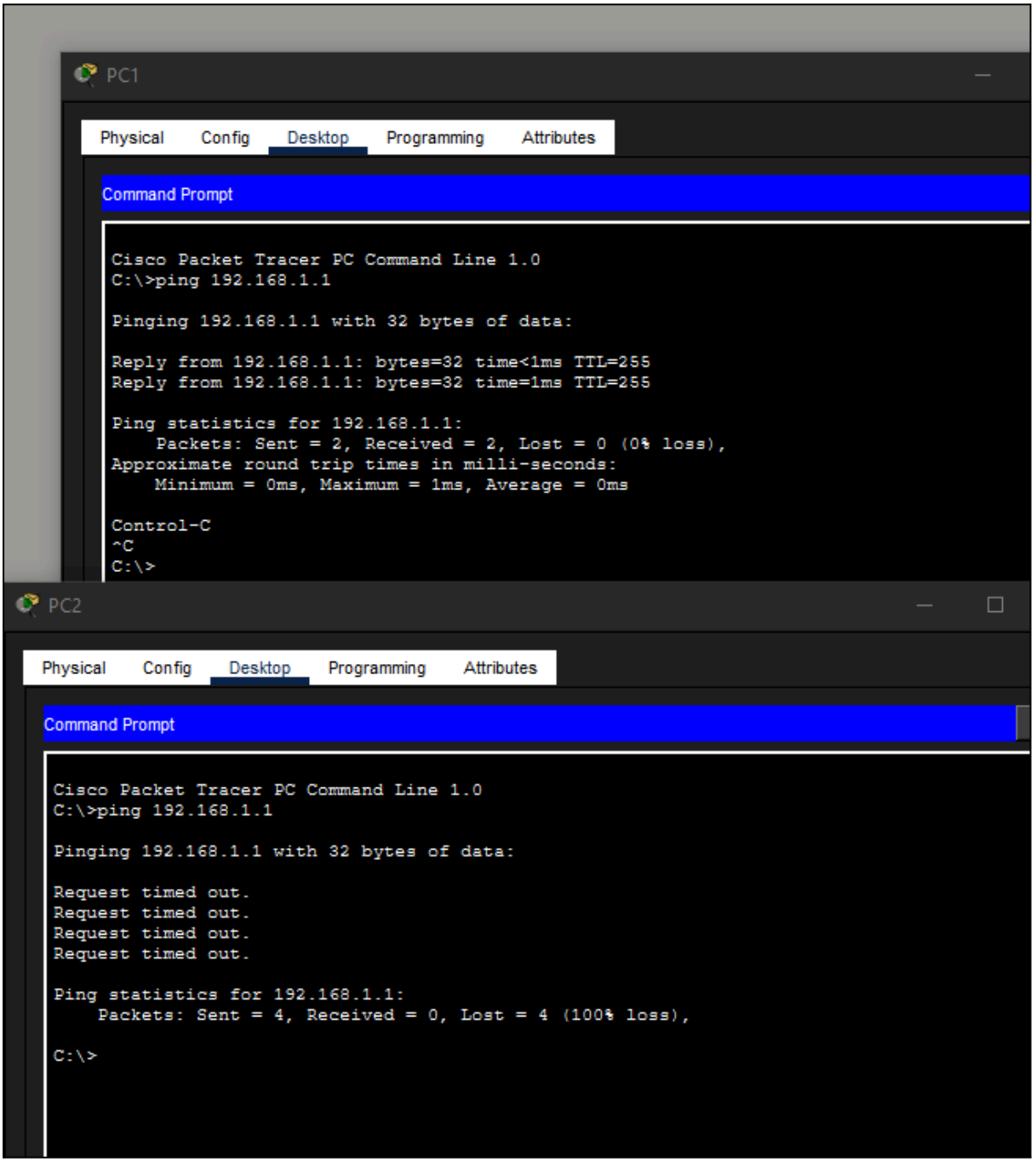
4. y 5. Configuración de la ACL y aplicación a una interfaz

```
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface gigabitEthernet0/0
Router(config-if)#access-list 1 deny 192.168.1.4
Router(config)#0.0.0.0 access-list 1 permit any
               ^
% Invalid input detected at '^' marker.

Router(config)#access-list 1 deny 192.168.1.4 0.0.0.0
Router(config)#access-list 1 permit any
Router(config)#inter
Router(config)#interface g0/0
Router(config-if)#ip access-group 1 in
Router(config-if)#
```

Creamos la ACL numerada 1 para bloquear la IP 192.168.1.4 (*access-list 1 deny 192.168.1.4 0.0.0.0*) y permitir el resto del tráfico (*access-list 1 permit any*). Después la aplicamos a la interfaz de entrada g0*/0* con *ip access-group 1 in*. El primer intento dio un error de sintaxis (Invalid input detected) porque la línea estaba mal escrita, y la repetimos bien.

6. Prueba de la configuración



Probamos que la ACL funciona con ping al router (192.168.1.1). Desde PC1 el ping responde con normalidad. Desde PC2, el equipo bloqueado, las cuatro peticiones caducan (Request timed out) y el resultado es un 100 % de paquetes perdidos, así que la ACL bloquea su tráfico y deja pasar el del resto, que es lo que buscábamos.

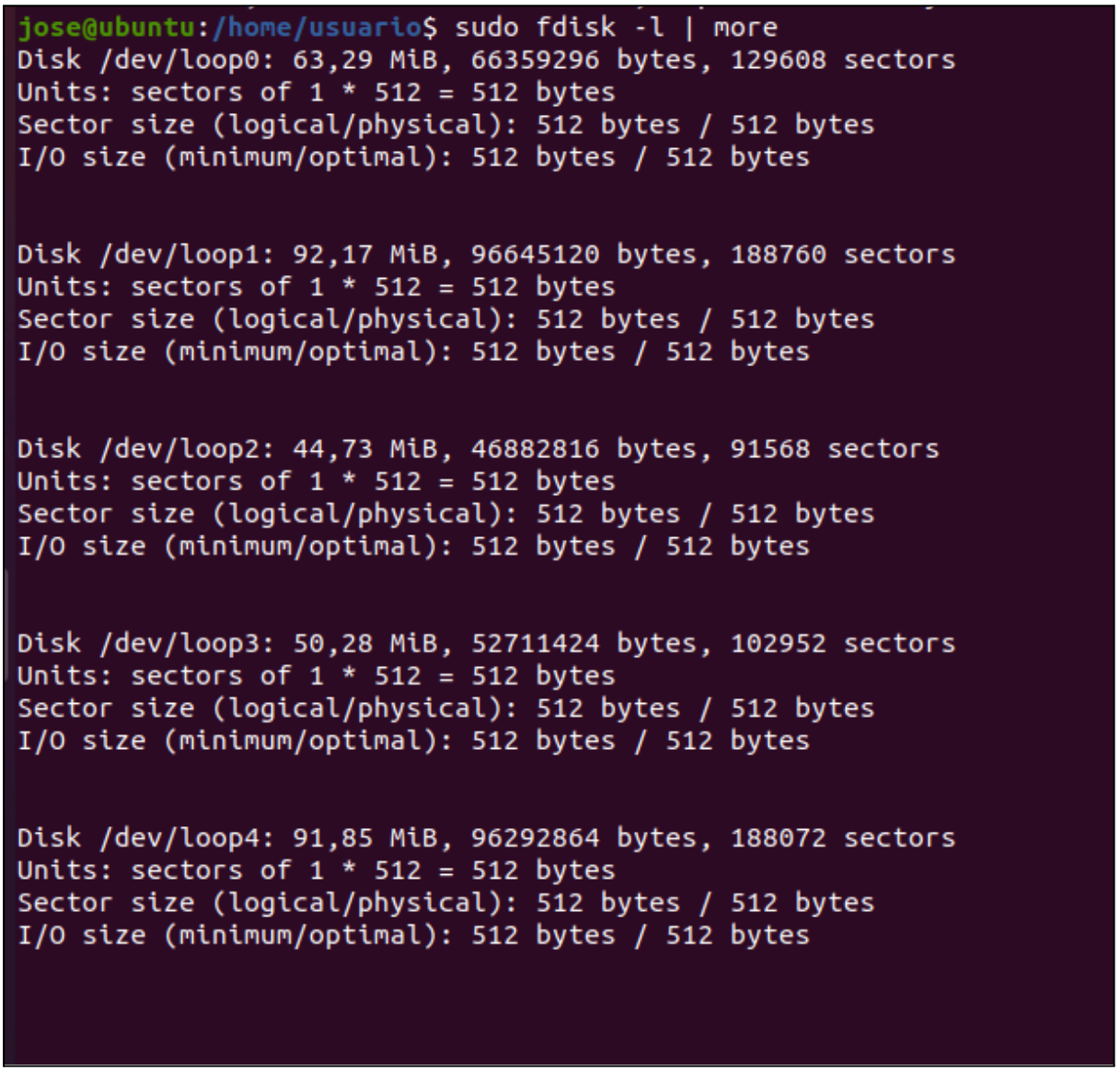
Práctica 5: Cómo montar un RAID 5 en una máquina virtual usando mdadm desde Ubuntu

2. Instalación de mdadm

```
jose@ubuntu:/home/usuario$ sudo apt install mdadm
[sudo] password for jose:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias
Leyendo la información de estado... Hecho
mdadm ya está en su versión más reciente (4.1-5ubuntu1.2).
fijado mdadm como instalado manualmente.
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 3 no actualizados.
jose@ubuntu:/home/usuario$
```

Instalamos mdadm con *sudo apt install mdadm*. En nuestro caso ya estaba instalado en su versión más reciente, así que no se instaló nada nuevo.

3. Identificar los discos



Listamos los discos con *sudo fdisk -l* | *more*. En esta primera pantalla solo salen los dispositivos loop del sistema; los discos nuevos del RAID los identificamos en la captura del paso siguiente.

4. Configuración de los discos

```
jose@ubuntu:/home/usuario$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.34).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS disklabel with disk identifier 0xba0ec648.

Command (m for help):


Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p):
```

Empezamos a preparar los discos nuevos con *sudo fdisk /dev/sdb*. Como el disco no tenía tabla de particiones, fdisk crea una nueva tabla DOS, y con la opción n empezamos a crear la partición (nos pregunta si será primaria o extendida).

```
jose@ubuntu:/home/usuario$ sudo fdisk -l /dev/sdb /dev/sdc /dev/sdd /dev/sde
Disk /dev/sdb: 25 GiB, 26843545600 bytes, 52428800 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/sdc: 25 GiB, 26843545600 bytes, 52428800 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/sdd: 25 GiB, 26843545600 bytes, 52428800 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/sde: 25 GiB, 26843545600 bytes, 52428800 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

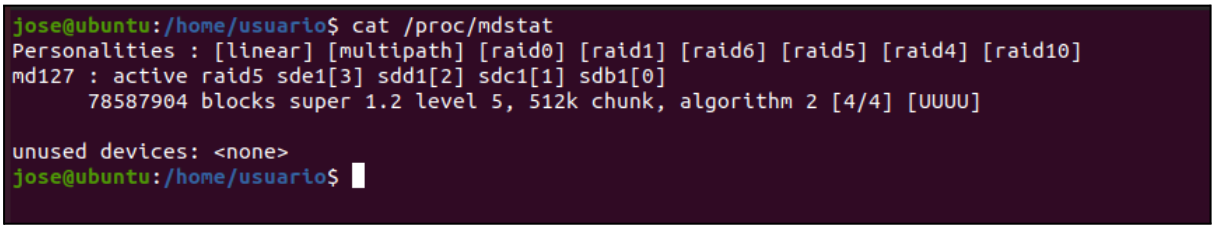
Con *fdisk -l* sobre los cuatro discos comprobamos que el sistema reconoce */dev/sdb*, */dev/sdc*, */dev/sdd* y */dev/sde*, todos de 25 GiB.

5. Configuración de mdadm



Creamos el RAID 5 con *sudo mdadm --create /dev/md/dm0 --level=5 --raid-devices=4* usando las particiones */dev/sdb1*, */dev/sdc1*, */dev/sdd1* y */dev/sde1*, con la opción *--assume-clean*. mdadm nos confirma que el array se ha iniciado.

6. Verificación del estado del RAID



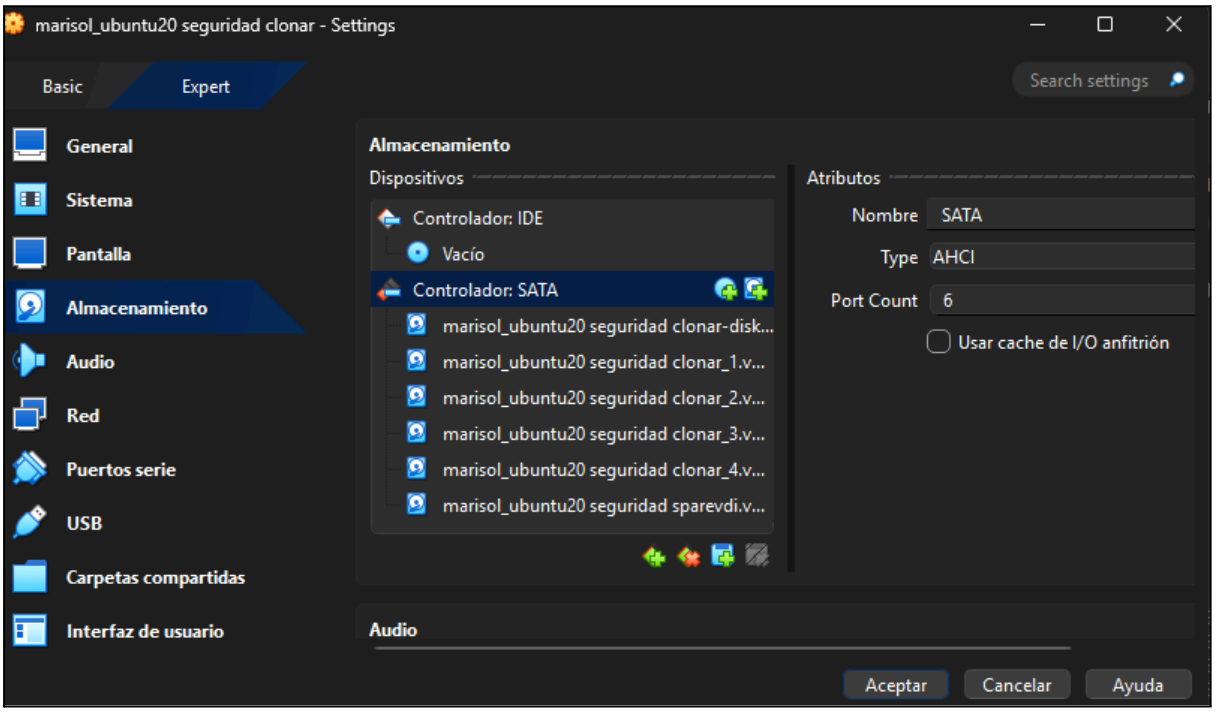
Con *cat /proc/mdstat* comprobamos que el RAID está activo: aparece como md127, de nivel raid5, con las cuatro particiones y el estado [UUUU], lo que significa que los cuatro discos funcionan bien.

7. Formateo del dispositivo lógico /dev/md/dm0



Formateamos el RAID con *sudo mkfs.ext3 /dev/md/dm0*. En la captura se ve cómo se crea el sistema de archivos ext3 con sus bloques, sus inodos y el journal.

8. Creación del disco spare



Para tener un disco de reserva (spare) añadimos un sexto disco virtual a la máquina desde la configuración de almacenamiento de VirtualBox. En el controlador SATA ya aparecen el disco del sistema, los cuatro discos del RAID y el disco spare (el puerto SATA pasa a tener 6). Este disco servirá para reemplazar a uno del RAID si falla.

9. Fallo de un disco del RAID

```
jose@ubuntu:/home/usuario$ sudo mdadm -D /dev/md127
/dev/md127:
           Version : 1.2
     Creation Time : Wed Nov  4 19:15:32 2026
        Raid Level : raid5
        Array Size : 78587904 (74.95 GiB 80.47 GB)
     Used Dev Size : 26195968 (24.98 GiB 26.82 GB)
      Raid Devices : 4
     Total Devices : 4
       Persistence : Superblock is persistent

       Update Time : Wed Nov  4 19:24:50 2026
             State : clean, degraded
    Active Devices : 3
   Working Devices : 3
    Failed Devices : 1
     Spare Devices : 0

            Layout : left-symmetric
        Chunk Size : 512K

Consistency Policy : resync

              Name : ubuntu:dm0  (local to host ubuntu)
              UUID : da61075b:542b43a2:fea0c5ae:aa04b035
            Events : 2

    Number   Major   Minor   RaidDevice State
       -       0        0        0      removed
       1       8       33        1      active sync   /dev/sdc1
       2       8       49        2      active sync   /dev/sdd1
       3       8       65        3      active sync   /dev/sde1

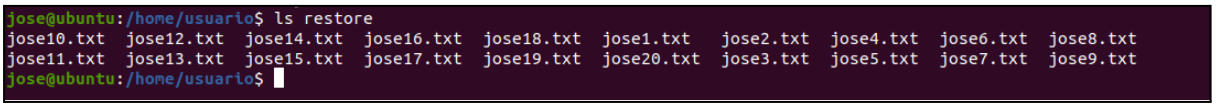
       0       8       17        -      faulty   /dev/sdb1
jose@ubuntu:/home/usuario$ cat /proc/mdstat
Personalities : [raid6] [raid5] [raid4] [linear] [multipath] [raid0] [raid1] [raid10]
md127 : active (auto-read-only) raid5 sdd1[2] sdb1[0](F) sdc1[1] sde1[3]
      78587904 blocks super 1.2 level 5, 512k chunk, algorithm 2 [4/3] [_UUU]

unused devices: <none>
jose@ubuntu:/home/usuario$
```

Provocamos un fallo en uno de los discos del RAID, el */dev/sdb1.* Con *sudo mdadm -D /dev/md127* vemos que el RAID pasa a estado clean, degraded: quedan 3 dispositivos activos y 1 fallido (aparece como faulty */dev/sdb1*). Después, con *cat /proc/mdstat*, el disco sale marcado con (F) y el estado del array pasa a *[_UUU]*. Como el sistema lo ha llamado */dev/md127*, usamos ese nombre en los comandos en vez de */dev/md0.*

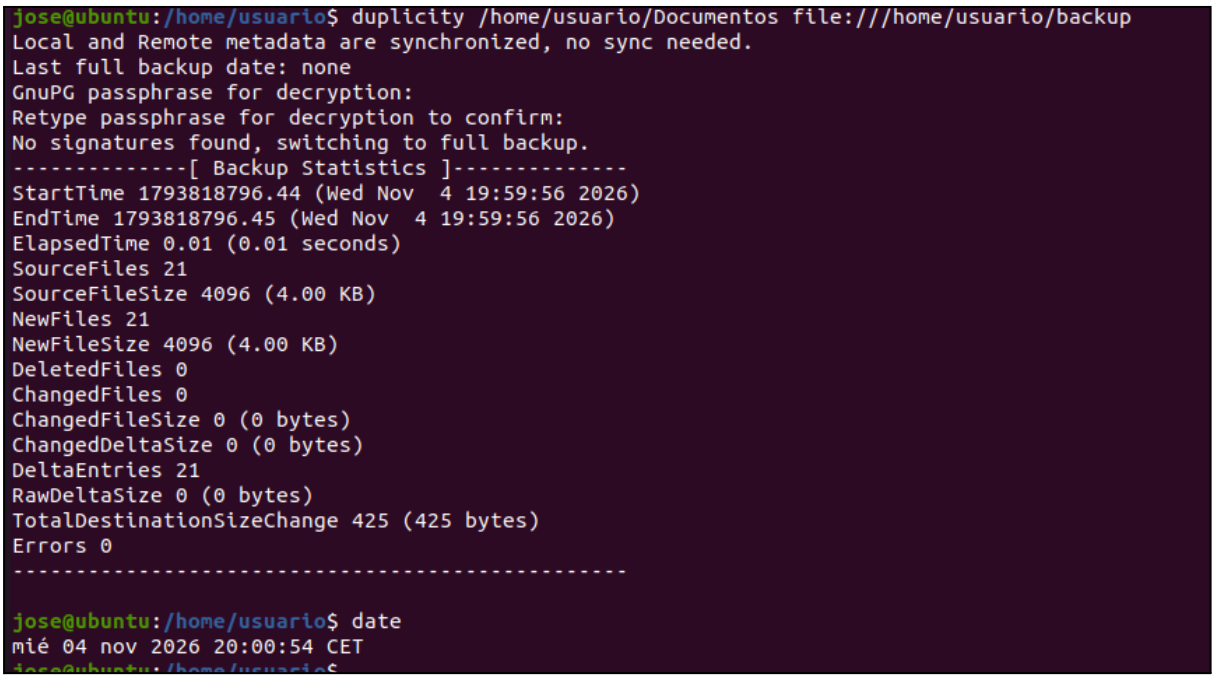
Práctica 6: Copias de seguridad con duplicity

2.1.- Creación de los archivos para realizar el backup



Estos 20 archivos los creamos dentro de la carpeta Documentos con el comando *touch jose{1..20}.txt*, que genera de una vez los archivos desde *jose1.txt* hasta *jose20.txt*, todos vacíos. Después los listamos con *ls* para comprobar que se habían creado bien. Son los archivos que luego usamos como datos para hacer la copia de seguridad con duplicity. PD: perdimos el comando al limpiar la terminal y se nos olvidó hacer la captura. Para no rehacer todo hemos empleado la misma captura que el 6.6

4.- Realización de la primera copia de seguridad COMPLETA



Hicimos la primera copia con *duplicity /home/usuario/Documentos file:///home/usuario/backup*. Nos pidió una contraseña para cifrar los datos y, como todavía no había ninguna copia anterior, duplicity avisó de que pasaba a hacer una copia completa (switching to full backup). En las estadísticas vemos 21 archivos de origen, 21 archivos nuevos y 0 errores. Debajo ejecutamos el comando *date*.



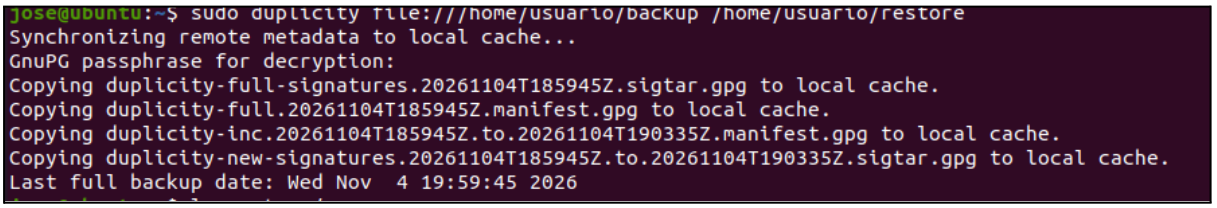
Entramos en la carpeta backup y listamos su contenido. Aparecen el manifest, el vol1.difftar y las signatures, todos con extensión .gpg (están cifrados) y con la palabra full en el nombre, que indica que es una copia completa.

5.- Creación de copias de seguridad INCREMENTALES

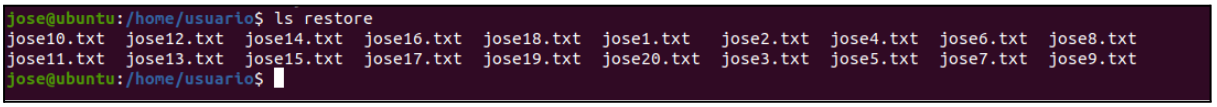


Hicimos la copia incremental añadiendo la opción incremental al comando de duplicity y después listamos la carpeta backup. Además de los archivos de la copia completa, ahora aparecen los de la incremental (inc y new-signatures), que solo guardan lo que ha cambiado desde la última copia.

6.- Restauración de datos

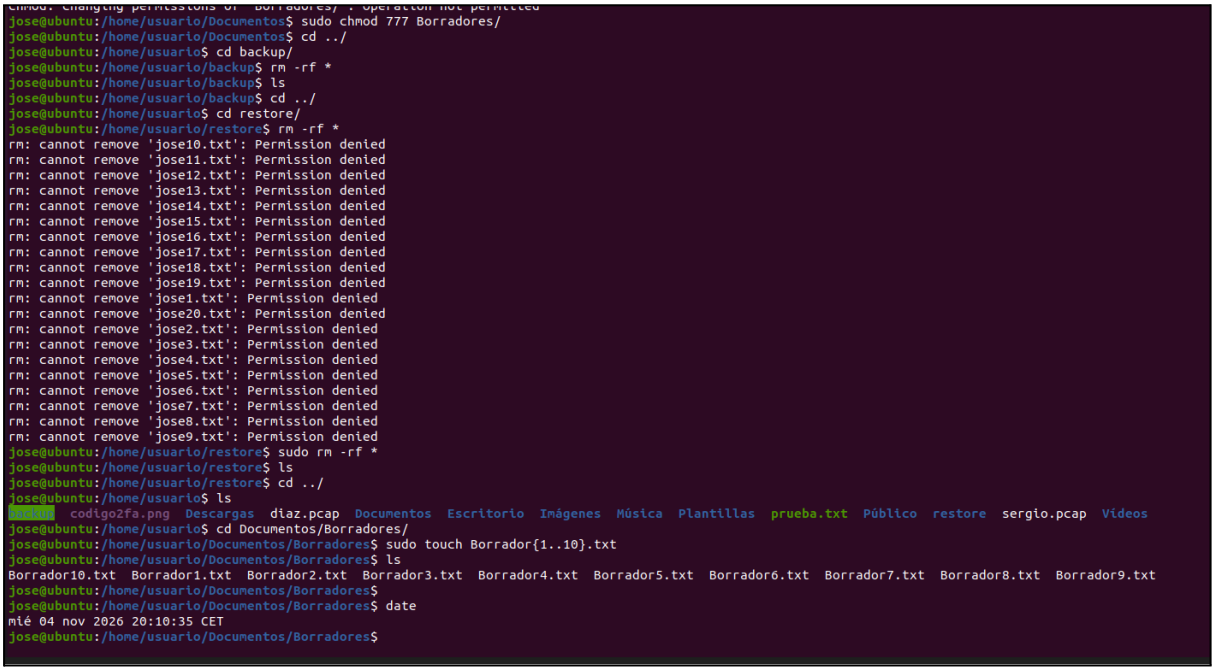


Para restaurar los datos ejecutamos *sudo duplicity file:///home/usuario/backup /home/usuario/restore*. Nos pide la contraseña de descifrado y duplicity copia las firmas a la caché local antes de restaurar.

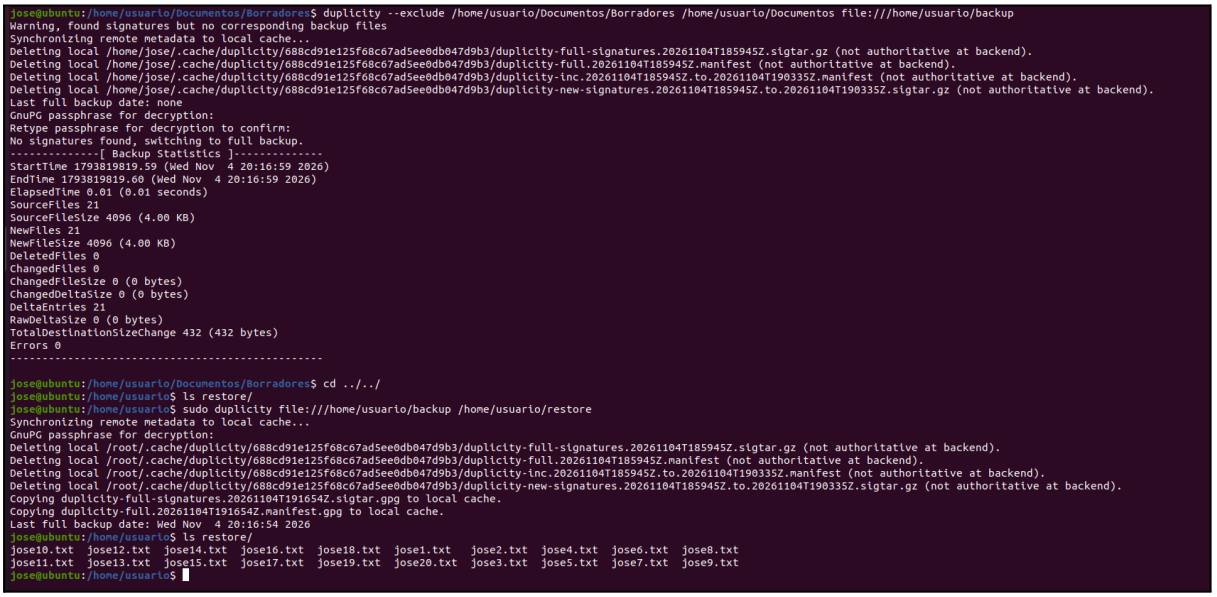


Con *ls restore* comprobamos que se han restaurado los 20 archivos, de *jose1.txt* a *jose20.txt*.

7.- Copias EXCLUYENDO el contenido



Antes de probar la exclusión vaciamos las carpetas backup y restore; en restore nos dio Permission denied, así que tuvimos que borrar el contenido con sudo. Después, dentro de Documentos*/Borradores*, creamos los diez archivos con *sudo touch Borrador[1..10].txt*, los listamos con *ls* y ejecutamos el comando *date*.



Hicimos la copia con la opción *--exclude /home/usuario/Documentos/Borradores*. Como habíamos vaciado la carpeta backup, duplicity la hizo como completa. Después restauramos con *sudo duplicity file:///home/usuario/backup /home/usuario/restore* y, al listar restore, solo aparecen los archivos *jose1.txt* a *jose20.txt*: no hay nada de la carpeta Borradores, así que la exclusión ha funcionado.